

Counting Before Calling: Receipt-Gated Analysis-Unit Accounting for a Stopped Multi-Agent Adjudication Study

Anonymous Author(s)

Abstract

A multi-agent evaluation can look adequately sized and still be far too small, because the things that are easy to count, programs, selected items, model calls, are not the units the planned statistic consumes. We report one such case from our own pipeline. A stopped adjudication study had prepared 101 clean and 69 faulty programs, but its estimand consumed matched task-level pairs, and only eight candidates for such pairs existed before human labeling. A receipt in the pipeline had passed a count of 51 selected items to the power calculation instead, which made the study look more than six times larger than it was. A human audit caught the error and stopped the study before any protected model call.

From that case we describe receipt-gated analysis-unit accounting, a workflow that types study units end to end so that tasks, programs, selected items, and eligible analysis units cannot be substituted for one another, and that gates every protected call on a deterministic adequacy check over the eligible units. We add a sensitivity analysis showing that the stop decision does not depend on the assumed effect size, discordance rate, or target power: across fifty combinations of the three, eight units are inadequate in every one, fifteen units in every one, and fifty units in all but seven. An executable validator now enforces the required receipt type at the pipeline’s run boundary. The evidence is a single retrospective case and synthetic validator fixtures, and we present it as such: a documented failure, the mechanism that would have caught it, and the attrition ratios a future study of this shape should plan for.

Keywords

multi-agent systems, study accounting, analysis units, progression criteria, test oracle problem

1 Introduction

What went wrong. We were building a study in which two agents would adjudicate disagreements between a candidate program and its oracle, with roles swapped so that each agent judged the other’s work. Each observation in that design is a compound unit: two separately constructed programs for the same task, a qualifying fault, a complete role swap, and an admissible gold label. None of the counts the pipeline produced on its way to that unit is interchangeable with it. We had 114 tasks, 228 accepted source programs, and after qualification and promotion 101 clean and 69 faulty programs. Task identity capped the mirror-capable tasks at fifteen, and eight mirror candidates existed before human labeling. A receipt upstream of the power calculation carried a count of 51 selected items rather than the eight candidates, and the calculation reported adequacy. A human audit of the preserved trail found the substitution, and the study stopped before its first protected model call. Nothing in the pipeline had detected it.

Why it matters beyond this study. Multi-agent evaluations plan from what is easy to count. Benchmarks report tasks, generators report programs, budgets report calls. When the planned statistic consumes a compound unit, each of those counts overstates the study, and the overstatement compounds through every filter. In our case the ratio was 50 to 15 to 8. A study that plans from the first number and reports the last has spent its budget before it learns that it cannot answer its question. Tools that record what happened, provenance systems, schema validators, experiment trackers, do not check that the declared analysis unit matches the data about to be consumed, and preregistration freezes the plan without checking the data against it. Section 2 places each of these against that requirement.

What this paper does. We describe the accounting workflow that would have caught the error and that now enforces the boundary. Study units are typed from construction through adjudication, gates emit typed receipts, and no protected call is made until a deterministic adequacy check over the eligible units passes. We validate the workflow three ways within the limits of a retrospective case. An exhaustive constrained-assembly audit establishes how many matched pairs the corpus could have yielded under every reservation and quota policy. A sensitivity analysis shows that the adequacy verdict for eight, fifteen, and fifty units is stable across fifty combinations of effect size, discordance rate, and target power. And an executable validator, tested on synthetic fixtures, rejects the historical receipt and every fixture that substitutes a count for a unit.

What it does not do. This is one case from one pipeline. The error was caught by a person, not by the validator, which was written afterward. We do not measure the validator on other pipelines, and we make no claim about the relative performance of the two adjudication protocols the study was designed to compare. Section 7.2 states these boundaries.

Contributions.

- A typed accounting model that records gate transitions without converting tasks, programs, selected items, and eligible analysis units into one another, with an executable validator at the adequacy boundary (Section 3).
- A preserved case: the 51-item substitution, its human detection, the stop, and the exact and sensitivity analyses of the adequacy verdict (Sections 5 and 6.5).
- The attrition ratios of a matched-pair multi-agent design, 50 to 15 to 8, as planning priors for studies of the same shape (Section 7).

2 Related Work

2.1 Multi-agent evaluation and controlled depth

ChatDev, MetaGPT, and AutoGen organize language-model calls into roles and conversations for software work [6, 15, 20]. Debate and independent sampling offer other paths from several judgments to one answer [4, 12]. Their results depend on orchestration as well as agent count. Equal thinking-token budgets can remove an apparent multi-agent advantage [18]. Trace studies locate failures in system design, inter-agent alignment, and task verification [3]. MAS-Orchestra treats orchestration choices as the experimental object [9].

Budget-controlled evaluation motivates the design. The planned protocols match call count and depth, while token use and latency remain measured rather than assumed equal. G3R and M4 each specify three parallel upstream calls, one shared deterministic executor, and one resolver. They use the same requested model and execution limits. The planned contrast concerns the complete protocols because routed views, specialist roles, and evidence flow change together.

2.2 Feasibility and progression criteria

Pilot and feasibility methodology distinguishes testing whether a definitive study can be done from testing the definitive effectiveness hypothesis. The CONSORT extension for randomized pilot and feasibility trials asks authors to state progression criteria and report how pilot evidence informs the future study [5]. The present setting is not a clinical trial. The relevant methodological principle is the separation of feasibility from a confirmatory claim. A locked progression rule should be evaluated on the unit used by the future analysis.

Software and agent evaluations often report substrate yield without an explicit adequacy gate. Our accounting workflow connects progression criteria to typed analysis units aligned to the estimand. Its postmortem executable gate refuses an untyped count before the main-call boundary.

Estimand frameworks emphasize aligning study design, data collection, and analysis with the exact target quantity [7]. Our concern is the upstream construction and accounting of the compound analysis units required to estimate that quantity.

2.3 Workflow provenance

in-toto records ordered supply-chain steps and verifies the materials and products associated with them [17]. RO-Crate packages research artifacts with structured metadata [16]. Workflow Run RO-Crate captures inputs, outputs, code, and execution context across workflow systems [11]. These systems motivate durable descriptions of what a stage consumed and produced. We do not claim content addressing or provenance as novel.

Provenance records what was consumed and produced. Our contribution is an analysis-unit accounting layer that types stage outputs, solves constrained assembly, and evaluates adequacy at the protected-call boundary. We do not claim the receipt substrate itself as new.

Table 1: Whether common tools enforce, before execution, that a declared analysis unit matches the data about to be consumed. Assessed from each tool’s documentation.

Tool	Enforces	What it does instead
in-toto attestations	partial	binds digests to predicates; a custom policy can gate on them
JSON Schema	partial	fixed structural constraints on one document
MLflow, Weights and Biases	no	record parameters, metrics, and artifacts
DVC	partial	detects changed inputs and dependencies
OSF preregistration	no	freezes the plan, read-only and time-stamped
CONSORT and SPIRIT	no	reporting checklists after the fact

2.4 Provenance, schemas, and preregistration

The requirement this paper enforces is narrow: before a protected call, the declared analysis unit must match the data that will be consumed. Table 1 asks of each common tool whether it meets that requirement, using the tool’s own documentation. Provenance attestations bind a subject digest to a predicate and can be placed as a gate, but the standard predicates say nothing about analysis units, so the check has to be written as a custom policy. A schema validator can enforce fixed structural constraints on one document, but it cannot compare a declared count with the eligibility of rows in another dataset. Experiment trackers record parameters and metrics and do not gate execution. Data versioning detects changed inputs, not a mismatch between a declaration and the meaning of the data. Preregistration freezes the plan and leaves the data unexamined. Reporting guidelines apply after the fact. The validator described here is the missing piece in that list, and it is small precisely because everything else in the list already exists.

The statistical side of the problem has its own literature. Card et al. document how underpowered NLP experiments are and argue for power audits [2]. Nosek et al. describe preregistration as a discipline for separating planned from post hoc analysis [14]. Kapoor and Narayanan catalog silent pipeline errors, leakage in particular, that are found only by later audit [8]. Bouthillier et al. account for the variance that benchmark designs ignore [1], and Ulmer et al. propose experimental standards that include adequacy and reporting gates [19]. The unit-substitution error we report is a member of the class Kapoor and Narayanan describe, and the locked progression rule is a preregistration of the adequacy decision rather than of the hypothesis.

3 Receipt-Gated Analysis-Unit Accounting

3.1 Typed study units

Let each stage operate on an explicit unit universe. The units used here are

$\mathcal{U}_T$	task identities,
$\mathcal{U}_K$	construction packets,
$\mathcal{U}_P$	accepted programs,
$\mathcal{U}_F$	frozen mutation candidates,
$\mathcal{U}_W$	witnessed fault candidates,
$\mathcal{U}_R$	reviewed programs,
$\mathcal{U}_I$	independently checked programs,
$\mathcal{U}_{QC}$	qualified clean programs,
$\mathcal{U}_{QF}$	qualified fault programs,
$\mathcal{U}_{MC}$	mirror-capable tasks,
$\mathcal{U}_M$	role-swapped mirror pairs,
$\mathcal{U}_S$	supplemental items,
$\mathcal{U}_A$	eligible primary analysis units.

A task is a stable benchmark identity. A construction packet is one isolated request for one lineage of one task. A program is accepted source from that packet. A frozen mutation candidate is selected without outcome access from an outcome-free ranked universe. A witnessed fault candidate carries the required reproducible divergence evidence. A reviewed program carries a completed panel verdict. An independently checked program carries a fresh-input check. A qualified clean program has passed the clean path. A qualified fault program has passed the fault, witness, and qualification-review requirements. A mirror-capable task has two separately constructed clean lineages and a qualifying fault in the required lineage. A mirror pair is the two role-swapped presentations assembled from those programs. A supplemental item belongs to a locked control or challenge stratum. An eligible primary analysis unit is a mirror pair that also survives every later eligibility requirement.

The distinctions make denominator conversion auditable. Two programs are not a task, two items are not a pair, and a pre-gold mirror candidate is not an eligible primary unit. Historically, however, the adequacy helper still accepted a bare integer, so these receipt fields did not enforce that distinction. Section 6 reports the later human audit and the postmortem executable check.

3.2 Typed gates and receipts

For stage s , a gate is an evidence-producing map over stage jobs

$$g_s : \mathcal{J}_s \longrightarrow \mathcal{U}_s \uplus \{\text{EXCLUDED, UNRESOLVED, INFRASTRUCTURE FAILURE}\}. \quad (1)$$

Each stage job $j \in \mathcal{J}_s$ references a finite predecessor set $P(j) \subseteq \bigcup_r \mathcal{U}_r$. A stage job may bind one or more predecessor units. Multiple jobs may reference the same predecessor, allowing branching, while a downstream job may bind several predecessors. Constrained dataset assembly remains a separate many-to-many operation.

The disjoint result states keep scientific exclusion separate from missing judgment and failed execution. A retry policy may resolve an infrastructure failure. It does not turn that state into an eligible unit without a recorded successful successor.

Every gate receipt records the stage-job identity, input unit types, output unit type, predecessor hashes, eligibility decision, reason, attempt status, and successor identity. The successor identity names the next unit when one exists. It also prevents a program, item, and pair from sharing one ambiguous identifier. Receipts are append-only study records. A correction writes a successor receipt that names the superseded record and preserves its bytes.

This contract supports local verification. It does not prove that an operator who controls the evidence root could not delete or replace the entire root. Without an external anchor, receipts are not tamper evidence. Hashes establish equality to preserved bytes and no more.

3.3 Constrained assembly

Let binary x_t indicate that task t supplies a mirror candidate. Let binary z_{tk} indicate assignment of task t to supplemental stratum k . The binary constants c_{t1} and c_{t2} mark qualified clean programs from distinct construction packets. The constants f_t and e_{tk} mark the required qualified fault and supplemental eligibility. Let R_D contain development identities reserved outside the main assembly. Let R_C contain the locked challenge reservation pool and let $F_C \subset R_C$ contain the challenge identities selected as a fixed main stratum. Variable mirror and supplemental assignments exclude $R_D \cup R_C$.

Let $\mathcal{I}(x, z)$ be the main-item set containing two presentations for each selected mirror task, one item for each selected supplemental task, and the fixed challenge items indexed by F_C . For task t and assignment a , the tie rank is the SHA-256 digest of canonical JSON containing the private selection seed, audit-case identity, task identity, and a . The vector $H(x, z)$ lists those ranks in locked task order and is compared lexicographically.

For a declared supplemental case, the selector first verifies that every active minimum stratum is satisfiable and restricts the domain to assignments that meet those minima. On that admissible domain it uses the lexicographic objective

$$\text{lexmax}_{x,z} \left(\sum_t x_t, \sum_{t,k} z_{tk}, -H(x, z) \right), \quad (2)$$

where L_k is the minimum for stratum k . The operational priorities are to maximize primary mirror tasks, require every active supplemental minimum, maximize supplemental items, and break remaining ties deterministically. Minimum satisfaction is an eligibility condition, not an objective term that becomes constant on the admissible domain.

The core constraints are

$$x_t \leq c_{t1}, \quad x_t \leq c_{t2}, \quad x_t \leq f_t, \quad (3)$$

$$z_{tk} \leq e_{tk}, \quad x_t + \sum_k z_{tk} \leq 1, \quad (4)$$

$$x_t = z_{tk} = 0 \quad \text{for } t \in R_D \cup R_C, \quad (5)$$

$$L_k \leq \sum_t z_{tk} \leq U_k \quad \text{for each active } k. \quad (6)$$

The two clean flags in Eq. 3 refer to different packet hashes and therefore different construction lineages. The fault flag includes the locked parent and witness requirements. Equation 4 keeps a task out of multiple strata. Equations 5 and 6 enforce development exclusion, challenge-pool reservation, and variable control quotas. The fixed challenge quota is $|F_C|$ and is checked before optimization. In the case study, $|R_C| = 10$ and $|F_C| = 8$.

Scheduling is a separate deterministic step, not a variable in Eq. 2. After allocation, the producer ranks item IDs by SHA-256 over a recorded seed, domain, item ID, and rejection counter. It takes the first counter below 10^6 whose order keeps mirror mates nonadjacent, then uses a separately domain-tagged rank to balance which arm runs first. The verifier reproduces both operations.

3.4 Adequacy and progression

Adequacy is evaluated at two separate gates, because the eligible-unit receipt set does not exist until pair-level final adjudication has run. Let δ be the recorded paired risk difference, d the discordance assumption, α the two-sided test level, and $1 - \beta$ the target power. The pre-gold gate is

$$A_{\text{pre}}(\bar{N}; \delta, d, \alpha, 1 - \beta) \longrightarrow \{\text{ENTER_GOLD}, \text{STOP}\}, \quad (7)$$

where $\bar{N}$ is a verified upper bound on the candidate primary units that final adjudication could certify. This gate exists to prove STOP. Its ENTER_GOLD outcome authorizes the adjudication stage and nothing else: $\bar{N}$ is never converted into $\mathcal{U}_A$, never supplies n_A , and never suffices to release a main call. When power is not known to be monotone over the attainable range, the gate maximizes power over every $n \leq \bar{N}$ and returns STOP when that maximum misses $1 - \beta$. The main-call gate is

$$A_{\text{main}}(R_A; \delta, d, \alpha, 1 - \beta) \longrightarrow \{\text{PROCEED_MAIN}, \text{STOP}\}, \quad (8)$$

where R_A is the canonical eligible-unit receipt set produced after final adjudication, whose inner receipts and outer set are both typed $\mathcal{U}_A$, and $n_A = |R_A|$. It refuses a bare integer and sets or receipts typed $\mathcal{U}_S$, $\mathcal{U}_{M_C}$, or $\mathcal{U}_M$. Before counting, it verifies schema, content hashes, the declared eligibility field, unique and canonically ordered unit identities, and one shared content-run binding across every receipt. For a study run it derives a manifest-bound inventory of candidate primary pairs; every eligible receipt must be a unique member of that inventory and match the pair's bound public-item and private-record commitments, and final adjudication may yield a strict subset. A PROCEED_MAIN receipt binds the exact R_A hash, content-run identifier, and analysis parameters; the supported postmortem run CLI recomputes that decision before invoking either runner entry point and admits only PROCEED_MAIN under the tooling-bound audited threshold of 103 units.

In this study $A_{\text{pre}}(15) = \text{STOP}$, because exact enumeration found every $n \leq 15$ inadequate and pair-level final adjudication could only reduce the count. The pre-gold gate therefore closed the path: adjudication was never entered, no R_A was produced, and A_{main} was never reachable. No historical $\mathcal{U}_A$ receipts exist, consistent with final gold never running, so historical records cannot authorize that CLI. A redesign would require a new protocol identity and is outside these two binary decisions.

Algorithm 1. Receipt-gated analysis-unit accounting

- (1) Declare the analysis unit required by the primary estimand and every predecessor unit type.
 - (2) Apply each gate job to a verified predecessor set and append one typed receipt for every eligible, excluded, unresolved, or failed result.
 - (3) Assemble candidate primary and supplemental units under the locked constraints using Eq. 2.
 - (4) Before final eligibility exists, evaluate A_{pre} on a verified candidate upper bound. Use that bound only to prove STOP; never convert it into R_A , and never read ENTER_GOLD as authorization for a main call.
 - (5) After final adjudication, validate the canonical R_A receipt set and evaluate A_{main} under the locked analysis with n_A .
 - (6) Write the progression decision. A correction names and preserves its predecessor in a successor receipt.
 - (7) Release main calls only after the runner recomputes a valid PROCEED_MAIN receipt bound to R_A . A stop at either gate retains the no-call boundary.
-

4 Case Study Design

4.1 Pair-level final adjudication

The motivating case asks which source is responsible when a candidate and a designated oracle disagree. Final decisions allow candidate-only fault, oracle-only fault, faults in both, neither faulty, or material specification ambiguity. A resolver may abstain. Transport, timeout, schema, and client failures are execution states rather than semantic labels. Material ambiguity applies only when plausible readings change program correctness and prevent a unique attribution.

The planned pair-level final-gold process would record specification status and the status of neutral Program A and Program B. Two reviewers would judge each packet while blinded to system outputs, program provenance, candidate and oracle roles, mutation history, mirror links, and the other reviewer's answer. Only axis disagreements would reach a third reviewer. This stage was not entered.

4.2 Matched-depth, four-invocation protocols

G3R assigns three separate generalists the specification and both role-labelled programs. A strict plurality supplies a diagnostic. One resolver would receive all reports and executor traces. M4 assigns one specification analyst, one candidate critic, and one oracle critic. The analyst sees no program. Each critic sees the specification and only the assigned program. A deterministic merge forms a provisional decision before one resolver sees both programs, all reports, and executor evidence.

Both arms specify the same requested model, three parallel upstream calls, one resolver, and the same deterministic executor contract. Each upstream role may propose at most three evidence slots and two executable claims. The executor applies no more than six claims per arm to both programs and records returns, exceptions, timeouts, and observation differences. Proposed expected values carry no truth status. Actual token use and latency would be measured rather than assumed equal.

The planned contrast bundles role specialization, view restriction, and evidence routing. It cannot isolate any one mechanism. No G3R or M4 main call was made, so the study supports no comparison between them.

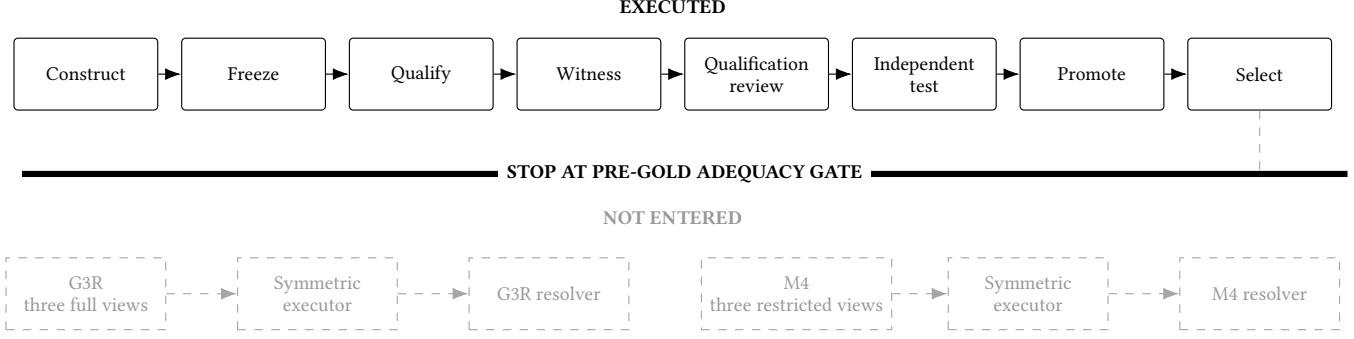


Figure 1: Executed preparation and the unentered comparison. Solid black gates produced verified receipts. Dashed gray protocol stages were never entered. The bold bar marks the preserved no-call boundary. The stop occurred at A_{pre} , so A_{main} and the pair-level final adjudication were never reachable.

4.3 Mirror analysis unit and original resource targets

Let C_A be one qualified clean program. Let D_B be a qualified fault derived from a second clean lineage C_B . One mirror candidate contains

$$I_1 = C_A \mapsto \text{Candidate}, \quad D_B \mapsto \text{Oracle}, \quad (9)$$

$$I_2 = D_B \mapsto \text{Candidate}, \quad C_A \mapsto \text{Oracle}. \quad (10)$$

The clean program paired with D_B is not its immediate parent. Mirror mates occupy nonadjacent contexts. Fault role and code-block order are balanced separately. Both-clean, both-faulty, and ambiguity items occupy task-disjoint supplemental strata.

Each selected main item would run both arms, and each arm contains three upstream model calls and one resolver call. The planned call count is therefore

$$C(I) = 2(3 + 1)|I| = 8|I|. \quad (11)$$

For arm a and task t , $Y_{a,t} = 1$ only when both presentations receive the exact gold label. The planned paired risk difference is

$$\Delta = \frac{1}{T} \sum_{t=1}^T (Y_{M4,t} - Y_{G3R,t}). \quad (12)$$

The locked analysis specifies an exact two-sided McNemar test at $\alpha = 0.05$ [13]. Power integrates the exact conditional rejection rule over the binomial discordance count [10]. The pre-output selection configuration records $\delta = 0.2$ in a field labeled “minimum detectable paired risk difference” and $d = 0.5$ as an assumed discordance rate; neither value was estimated from main outcomes, and the record gives no pilot estimate, citation, or other rationale for choosing them. The helper parameterizes cells as $p_{10} = (d + \delta)/2$ and $p_{01} = (d - \delta)/2$, with $|\delta| \leq d$. Historical protocol records name 60 mirror tasks as the original resource target and 40 as the minimum resource tier. The original target receipt remains unchanged.

Equation 11 resolves the call-count chronology. The historical selection first produced $|I| = 51$, after which its receipt derived $C = 408$. The selection configuration instead recorded 1,184 calls of remaining capacity, so 408 was not a locked budget. Likewise, the later 63-item re-audit reuses the executed supplemental counts

26 and 1, which are realized counts rather than quotas. They are not the same call count and not the internally recorded 10/10 supplemental target.

A pipeline receipt links construction, freeze, qualification, witness generation, qualification review, independent testing, promotion, and selection to verified predecessors. The anonymous artifact preserves those records and the protocol materials.¹

The source universe followed an explicit exclusion rule over the pinned 164-task HumanEval+ v0.1.10 release. From those 164 canonical identities we removed the 40 formative Study 0 tasks, taken deterministically as the first 40 of the every-fourth-task sequence in ascending canonical-ID order, and the ten identities reserved for the recorded live development run. The two excluded groups are disjoint from each other and from the retained set, leaving $164 - 40 - 10 = 114$ tasks, all of which received both construction jobs. The rule was fixed before any clean-construction response was observed and did not depend on mutation yield, execution outcomes, or reviewer-supplied fresh inputs, which entered only downstream qualification. The records do not document why those ten identities were reserved, so that exclusion remains a limitation.

5 Executed Pipeline and Attrition

No quantity in this section comes from a G3R versus M4 main run. Execution ends after protected-output-free selection. Table 2 reports the completed gates.

All 6,674 qualification jobs completed on the first attempt. The 39 clean-suite disagreements comprised seven base and 32 withheld-plus suite outcomes across 32 programs; seven programs disagreed on both suites. The locked screen admitted a program to qualification review only if it matched the designated reference on both suites. This was a conservative eligibility rule, not a correctness judgment, and can exclude a correct alternative implementation whose permitted output differs from the reference. Of 692 mutation survivors, 58 had an ineligible clean parent; 217 of the 634 witness suites yielded a returned-output witness. The fixed per-parent cap retained 81 fault candidates without consulting human verdicts.

¹<https://anonymous.4open.science/r/masgain2026-artifact>

Table 2: Executed preparation flow and stopped comparison boundary

Stage	Locked input	Executed units	Recorded disposition
Construction	114 tasks	228 jobs and 230 physical attempts	228 accepted source programs with one infrastructure retry and one quota pause
Mutation freeze	228 parents	6,218 generated mutations and zero executions	Outcome-free ranked universe
Qualification	456 clean suites and 6,218 mutation suites	6,674 terminal jobs	417 clean-suite agreements, 39 clean-suite disagreements, 692 mutation survivors, and 5,526 detections
Witness	692 survivors	634 terminal suites	58 were excluded. 217 suites yielded a returned-output witness and 417 did not
Qualification review	196 machine-clean programs and 81 fault candidates	277 double-reviewed neutral packets	275 primary verdict agreements and two resolved disagreements
Independent test	180 clean-program suites	180 jobs with two fresh cases each	179 agreements and one disagreement
Promotion	Verified stage lineage	277 reviewed programs	101 clean and 69 fault programs qualified. Attrition included 18 plan exclusions, 88 duplicate-source exclusions, and one independent non-pass
Selection and boundary	Verified promoted pool	51 selected items and eight pre-gold mirror-task candidates	Zero pair-level final-gold records and zero of 408 derived main model calls

Qualification reviewers agreed on 275 of 277 verdicts, for observed agreement 0.9928 and Cohen’s $\kappa = 0.9831$. The label matrix, read as first-reviewer rows against second-reviewer columns over correct, faulty, and ambiguous, was 193, 0, 0; then 1, 80, 1; then 0, 0, 2. Final panel labels were 193 correct, 80 faulty, and four materially ambiguous. Both disagreements concerned separately constructed implementations of one median task; the resolver classified both as materially ambiguous.

The independent stage scheduled 180 programs with correct panel verdicts and two distinct fresh inputs. One program disagreed with the designated reference. For HumanEval/89 and input Az-9_q, the program returned Ed-9_u while the designated reference returned Ad-9_u. The protocol retained this single-case non-pass for investigation. The disagreement does not establish that the designated reference is correct.

Promotion accounted for every reviewed program. The 277 decisions yielded 170 qualified programs and 107 explicit attritions. The attritions comprised 18 qualification-review plan exclusions, 88 duplicate-source exclusions, and the independent non-pass. The plan exclusions comprised 13 packets without distinct independent inputs, four material-ambiguity verdicts, and one clean candidate judged faulty.

The 101 clean and 69 fault programs did not form enough task structures. The resource ceiling in Table 3 ignores task identity and therefore overstates the usable supply. The identity ceiling requires two clean lineages and a qualifying fault within one task.

The resource ceiling is 50. Task identity lowers that ceiling to 15. The executed historical search maximized total items and selected 51 items containing eight pre-gold mirror-task candidates. This objective required a separate audit because maximizing all items does not establish the largest feasible mirror count.

6 Method Validation

6.1 Preserved unit-accounting correction

A retrospective audit of the preserved receipt trail found a denominator error while the prospective no-call boundary remained closed. One superseded selection receipt supplied the 51-item design-wide count to the paired power calculation. The primary analysis operates on mirror tasks, not all selected items. The

correction in Table 5 is a direct validation of typed analysis-unit accounting.

The preserved receipts did not autonomously detect the denominator error. They retained the unit identities, predecessors, and assumptions needed for a later audit to detect it, reproduce its consequence, and issue a successor record without erasing the historical one. The validator in Table 5 was implemented after that post-mortem. It makes zero model calls and reads no protected output, so it neither crosses nor alters the study’s stop decision. It rejects a bare integer and $\mathcal{U}_S$, $\mathcal{U}_{M_C}$, or $\mathcal{U}_M$ where R_A is required, and makes the supported run CLI refuse execution without a PROCEED_MAIN receipt exactly bound to the set, analysis, and content run. The corrected historical count of eight is a pre-gold upper bound proving $A_{\text{pre}}(8) = \text{STOP}$, not a relabeled $\mathcal{U}_A$; the frozen-pool audit later strengthened this to $A_{\text{pre}}(15) = \text{STOP}$.

6.2 Selector objective audit

The pilot selector maximized total selected items rather than primary mirror tasks. We therefore reran allocation with Eq. 2 and no model calls. The audit considered the mirror ceiling without supplemental constraints, with one item per supplemental stratum, under the executed supplemental counts, and under those counts plus an explicit 408-call constraint. Table 6 reports the allocations. Fixed development and challenge reservations remain unchanged.

The solver was exhaustive dynamic programming, not MILP or CP-SAT. The 41 flexible tasks comprised 26 eligible only for both-clean, six for both-clean or mirror, and nine for all three variable strata, giving $2^{26}3^64^9 = 12,824,703,626,379,264$ raw assignments. The DP visited 3,750 reachable count states and 89,618 assignment transitions, keeping the lexicographically least hash vector per state; the exact 26/1 case visited 837 states and 32,083 transitions. Exhaustion establishes the reported 15 and 14 optima. The historical receipt did not record elapsed time. A five-repeat solve-only rerun of the three cases without the call cap plus the recorded 10/10 and 8/8 diagnostics had median 4.210 seconds, with range 3.694–4.953 seconds. The 408-call row is arithmetic rather than a timed solver case.

The executed count of eight was not primary-unit-maximal absent a call cap. It resulted from a sequential objective that maximized total items first. The unit-first audit found 14 mirror tasks

Table 3: Attrition from promoted programs to candidate primary units

Stage	Program or item supply	Primary-unit status	Retention/ conversion factor
Promotion pool	101 clean and 69 fault programs	Primary units not yet assembled	Reference supply
Resource ceiling	Same promoted pool	At most 50 pairs from program counts alone	$\min(\lfloor 101/2 \rfloor, 69) = 50$
Identity ceiling	Same promoted pool	15 mirror-capable tasks	$15/50 = 0.30$
Historical total-item-first selection	51 main items with 16 mirror presentations	8 pre-gold mirror candidates in the historical allocation	$8/15 = 0.53$
Corrected unit-first selection under executed supplemental counts 26/1	63 potential main items with 28 mirror presentations	14 pre-gold mirror candidates in the re-audit	$14/15 = 0.93$
Original resource tiers	Historical allocation of eight candidate pairs	Insufficient for both historical tiers	$8/40 = 0.20$ and $8/60 = 0.13$

Table 4: Decision chronology by recorded date; rows sharing 2026-07-19 are not ordered within that day

Point	What was known	Criterion applied	Decision
Initial plan, 2026-07-15	No main output; target 60 and minimum 40 mirror tasks; $\delta = .2, d = .5$ recorded as design assumptions	Eligible mirror tasks, exact two-sided McNemar test, target power .8, and non-power gates	Prepare the substrate; keep final gold and main calls closed
Historical selection, 2026-07-18	101 clean and 69 fault programs; 51 items containing eight mirror candidates	Total-item-first allocation; superseded adequacy path accepted the bare count 51	Preserve the protected-output-free allocation; make no main call
Unit correction, dated 2026-07-18; successor 2026-07-19	Corrected deviation records eight pairs; successor preserves the prior 51-unit receipt	Eight mirror candidates; power .0158 and fewer than 40 task units	STOP before pair-level final gold and main calls
Selector re-audit, 2026-07-19	Identity ceiling 15; promoted pool and reservations unchanged	Exhaustive primary-unit-first allocation	14 under the executed supplemental counts, 15 without minima, and 8 under $C \leq 408$; retain STOP
Power audit, 2026-07-19	The original resource tiers had not reproduced target power	Separate exact enumeration under the recorded assumptions	60 gives .525; 103 first reaches .8; preserve the plan and its correction
Postmortem implementation, 2026-07-20	No historical $\mathcal{U}_A$ receipt exists	Accept only a hash-valid R_A and require its bound PROCEED_MAIN receipt	Add typed validator and runner gate; zero model calls and no protected-output read

Table 5: Postmortem receipt interface and historical negative control

```

Schema fragment
schema: masgain-v2-typed-unit-receipt-set/v1
unit_type: U_A
receipts[*]: {unit_type: U_A, unit_id,
  eligibility_decision: eligible, predecessor hashes,
  receipt_sha256}; receipt_set_sha256

Reproduction output
NEGATIVE_CONTROL historical_51: REJECTED
E_UNIT_TYPE_MISMATCH: adequacy requires
receipt_set<U_A>; received
masgain-v2-prepared-selection-receipt/v2
with 51 mixed selected main items

```

Table 6: Lexicographic selector re-audit

Constraint case	Mirror	Both- clean	Both- faulty	Items	Calls
No supplemental minima	15	26	0	64	512
At least one per stratum	14	26	1	63	504
Executed supplemental 26/1	14	26	1	63	504
Executed 26/1 and $C \leq 408$	8	26	1	51	408

Ten development identities remain excluded. Eight challenge items from a ten-identity pool are fixed. Calls equal eight times items.

under the executed supplemental counts of 26 and 1 and 15 when supplemental minima were removed. With 26/1 and eight fixed challenge items, $C \leq 408$ implies $8(26 + 1 + 8 + 2 \sum_t x_t) \leq 408$, hence at most eight mirrors; the historical allocation attains that bound. The call cap alone would still admit all 15 mirrors because their 30 presentations and eight challenge items require 304 calls. The pre-output supplemental target was 10/10 and the minimum was 8/8, not 26/1. The target was infeasible because only nine tasks

were both-faulty-capable; the minimum supported at most seven mirrors.

This correction does not reopen the study. Even the counterfactual pre-gold identity ceiling of 15 mirror tasks has power 0.098 under the locked analysis. The audit matters because a claim of maximality must follow the objective for the primary unit.

6.3 Executable validator checks

Final adjudication never ran, so the study produced no $\mathcal{U}_A$ receipt set. Every positive and tamper case in Table 7 is therefore an explicitly synthetic, non-study validator fixture rather than historical evidence. Only the first row is historical: the preserved superseded receipt, used as a negative control. The two positive fixtures are evaluated under a test-only eligible-unit threshold of two, while the real study-run CLI stays bound to the audited threshold of 103 units. The artifact ships every input and one command that checks the exact exit code and output for all seven cases.

6.4 Independent adequacy audit

The repository statistics helper recomputed unconditional exact two-sided McNemar power under $\delta = 0.2$, $d = 0.5$, and $\alpha = 0.05$. Table 8 reports rounded paper values. Exact floating-point values remain in the successor receipt and artifact. A separately implemented exact enumeration, written without importing the repository helper, reproduced all reported values. The repository preserves both implementations; the anonymous artifact preserves the independent enumerator and records both hashes.

The separate implementation directly enumerates the trinomial task outcomes, while the repository helper conditions on the number of discordant pairs. Both reproduced power of 0.0158 for eight

Table 7: Historical negative control and synthetic, non-study validator fixtures

Evidence scope	Input	Observed result
Historical negative control	Preserved prepared-selection/v2 receipt containing 51 mixed main items	REJECTED, E_UNIT_TYPE_MISMATCH
Synthetic, non-study	Valid $\mathcal{U}_A$ set with $n_A = 1$ under a test-only threshold of two	VALIDATED, STOP
Synthetic, non-study	Valid $\mathcal{U}_A$ set with $n_A = 2$ under the same test-only threshold of two	VALIDATED, PROCEED_MAIN
Synthetic, non-study	One-character inner content-hash tamper	REJECTED, E_RECEIPT_CONTENT_HASH_MISMATCH
Synthetic, non-study	Duplicate unit identifier with a recomputed outer hash	REJECTED, E_DUPLICATE_UNIT_ID
Synthetic, non-study	Noncanonical unit ordering with a recomputed outer hash	REJECTED, E_NONCANONICAL_ORDER
Synthetic, non-study	Intact PROCEED_MAIN receipt replayed against another content-run binding	REJECTED, E_CONTENT_RUN_BINDING_MISMATCH

Table 8: Exact-test power

Mirror pairs	Rounded exact-test power
8	0.016
15	0.098
40	0.354
60	0.525
103	0.802

historical candidates and found that the original 60-pair resource target would provide only 0.525 power under the stated alternative. We preserve the original target receipt and record this correction in a successor adequacy receipt. The first integer sample size that reaches 0.8 power is 103 pairs. Section 6.5 reports the full sensitivity grid.

The correction also calibrates the historical tiers. At target power 0.8, the minimum detectable risk difference is about 0.317 with 40 pairs and 0.264 with 60 pairs. Neither tier supports the pre-specified effect of 0.2 at target power. No feasible risk difference under the assumed discordance could attain 0.8 power with eight task-level units. The discordance-bound maximum is 0.145.

The observed supply chain was therefore 50 pairs under a resource-only count, 15 mirror-capable tasks under identity, and eight mirror-task candidates under the executed selection. The re-audit found 14 under the executed supplemental counts. Pair-level final adjudication could only decrease either candidate count. Even the identity ceiling remained inadequate, so A_{pre} returned STOP before final gold and before every planned main call.

6.5 Sensitivity of the adequacy verdict

The adequacy check in the case used a paired risk difference of $\delta = 0.2$ and a discordance rate of $d = 0.5$ under an exact two-sided McNemar test at $\alpha = 0.05$. Neither value is derived from data, so the natural question is whether the stop depended on them. We recomputed the verdict for every combination of $\delta \in \{0.10, 0.15, 0.20, 0.25, 0.30\}$, $d \in \{0.30, 0.40, 0.50, 0.60, 0.80\}$, and target power $\{0.80, 0.90\}$, using the published verifier without modification, for the three unit counts that appear in the case: the eight pre-gold mirror candidates that existed, the fifteen mirror-capable tasks that task identity allowed, and the fifty of the resource ceiling.

Table 9 gives the result. Eight units are inadequate under every combination, and so are fifteen. Fifty units become adequate only when the assumed effect is large, $\delta \geq 0.25$, and discordance is low, and even then in seven of fifty cells. At the recorded assumptions

Table 9: Adequacy verdicts across fifty combinations of effect size, discordance rate, and target power. A cell is adequate when exact power at that unit count reaches the target.

Units	Adequate	Inadequate	Where adequate
8	0	50	nowhere
15	0	50	nowhere
50	7	43	$\delta \geq 0.25$ with low discordance: four cells at power 0.8, three at 0.9

the required count is 103 units for 0.8 power and 135 for 0.9, and the maximum power attainable with eight units is 0.145. The stop therefore did not rest on the particular values chosen. Any defensible pair of assumptions leads to the same decision for the units this study had, and the grid also says what a successor would need: a design that expects a moderate effect and typical discordance needs on the order of a hundred matched pairs, which at this study’s yield of eight candidates from 114 tasks means starting from on the order of 1,400 tasks.

7 Discussion and Limitations

7.1 Compound attrition as a design parameter

The relevant yield in this study was not programs per task. It was complete evidence structures per eligible primary analysis unit. A mirror needed two accepted clean lineages, one witnessed fault, a valid role swap, and later pair-level final adjudication. Both-faulty controls needed faults under distinct parents. Review and fresh-input requirements removed programs that had passed machine suites. These dependencies made task-level yield far lower than program-level yield.

Future agentic studies should estimate each conversion before fixing a confirmatory design. The sequence is to declare the analysis unit aligned to the estimand, record program-level retention, assemble task structures under all reservations, and compute power from the eligible compound units. A small construction pilot can estimate these rates without entering the main comparison.

The adequacy correction adds a second lesson. A resource target is not a power target until the stated test, effect, discordance, and sample size reproduce the desired power. A successor receipt can repair the operational plan while retaining the historical record needed to audit the change.

7.2 Scope and interpretation

The executed stages establish an inspectable path from construction through selection. They support claims about gate behavior,

attrition, accounting, and the stop boundary. They do not support claims about G3R, M4, routed evidence, specialist roles, generalist review, or comparative protocol performance.

The construction universe contains public HumanEval+ tasks and a bounded mutation family. The construction model’s training corpus is unavailable, so prior HumanEval exposure cannot be ruled out and may affect yield. Generalization to production code, other languages, or natural defects is unknown. The two lineages used isolated sessions and distinct prompts but the same model family and style contract, so their errors may remain correlated.

Qualification review is fallible. Neutral packets, separate answers, fresh inputs, and resolution make decisions inspectable. Residual judgment error remains possible. A disagreement with the designated reference remains a prompt for investigation until the specification settles correctness.

The exact power calculations depend on the pre-specified risk difference and discordance. They diagnose the recorded design under its own assumptions. They do not establish power under every alternative analysis. The selector audit is also conditional on the frozen promoted pool, eligibility rules, reservations, and quota definitions.

The planned treatment changes role specialization, view partitioning, and evidence routing together. Even an adequately powered future run would not isolate those mechanisms. Token use, latency, abstention, and system failures remain unmeasured because the main protocols were not entered.

Original raw model streams and human administrative records remain outside the anonymous repository, which limits third-party replay of the private evidence chain. The evidence root remains local and under operator control, and a motivated operator could delete or replace it, so the receipts provide no tamper evidence against that operator and the study used no external anchor. Content hashes establish equality to preserved bytes, not study judgment. The validator checks the type, declared eligibility, and self-consistency of the supplied R_A and authenticates declared eligible-unit membership against local manifest commitments. It does not dereference predecessors, establish complete terminal-disposition coverage, open protected gold, or prove adjudication correctness.

One case, caught by a person. Everything in this paper rests on a single retrospective case from our own pipeline. The substitution was found by a human audit, and the validator that now rejects it was written afterward and tested on synthetic fixtures and on the historical receipt. We have not run it against another pipeline, and we do not know how often the error occurs elsewhere. What generalizes is the shape of the failure, a count standing in for a compound unit, and the shape of the check, a typed receipt that refuses the substitution. What does not generalize is any estimate of prevalence or of the validator’s effectiveness in the field.

Adoption cost. The validator requires a study to declare its analysis unit as a type and to route receipts through one CLI. We did not measure the effort of adding those declarations to a pipeline that lacks them, and we report that plainly rather than estimate it.

The replication package. The package released with this version carries the executable validator with its receipt-schema fragment,

sixteen unit tests, and a reproduction script that rejects the historical 51-item receipt, the independent power verifier, the repository statistics helper with its tests, the selector audit tool for source inspection, and the sensitivity script with its stored output. It does not carry the selector’s pinned candidate set or its timing logs, the benchmark snapshot with exact task identities, per-packet primary labels, or raw prompts, model outputs, and reviewer identities; the package README lists each omission with its reason.

8 Conclusion

Receipt-Gated Analysis-Unit Accounting makes compound-unit attrition explicit before a confirmatory comparison begins. In this case, 101 qualified clean programs and 69 qualified fault programs yielded 15 mirror-capable tasks. The historical total-item-first allocation held eight pre-gold mirror candidates, and the corrected unit-first re-audit found 14 under the executed supplemental counts of 26 and 1. Even the 15-task identity ceiling remained far below adequacy. The receipt trail enabled a human audit to detect and correct the error before protected outputs existed, but did not catch it automatically. The postmortem validator now rejects the wrong receipt type and requires a bound `PROCEED_MAIN` receipt at the supported run CLI. The original 60-pair resource target supplied only 0.525 power under the recorded assumptions, and every planned main call remained closed. No protocol-performance conclusion follows.

References

- [1] Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Nazanin Mohammadi Sepahvand, Edward Raff, Kanika Madan, Vikram Voleti, Samira Ebrahimi Kahou, Vincent Michalski, Tal Arbel, Chris Pal, Gael Varoquaux, and Pascal Vincent. 2021. Accounting for Variance in Machine Learning Benchmarks. *Proceedings of Machine Learning and Systems* 3 (2021). https://proceedings.mlsys.org/paper_files/paper/2021/hash/0184b0cd3c3fb185989f858a1d9f5c1eb-Abstract.html
- [2] Dallas Card, Peter Henderson, Urvashi Khandelwal, Robin Jia, Kyle Mahowald, and Dan Jurafsky. 2020. With Little Power Comes Great Responsibility. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 9263–9274. doi:10.18653/v1/2020.emnlp-main.745
- [3] Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why Do Multi-Agent LLM Systems Fail? *arXiv preprint arXiv:2503.13657* (2025).
- [4] Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2024. Improving Factuality and Reasoning in Language Models through Multiagent Debate. In *International Conference on Machine Learning (ICML)*. arXiv:2305.14325.
- [5] Sandra M. Eldridge, Claire L. Chan, Michael J. Campbell, Christine M. Bond, Sally Hopewell, Lehana Thabane, Gillian A. Lancaster, and PAFS Consensus Group. 2016. CONSORT 2010 Statement: Extension to Randomised Pilot and Feasibility Trials. *BMJ* 355 (2016), i5239. doi:10.1136/bmj.i5239
- [6] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xianwu Zheng, Yuheng Cheng, Jinlin Wang, Ceyao Zhang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2024. MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework. In *International Conference on Learning Representations (ICLR)*. arXiv:2308.00352.
- [7] Brennan C. Kahan, Joanna Hindley, Mark Edwards, Suzie Cro, and Tim P. Morris. 2024. The Estimands Framework: A Primer on the ICH E9(R1) Addendum. *BMJ* 384 (2024), e076316. doi:10.1136/bmj-2023-076316
- [8] Sayash Kapoor and Arvind Narayanan. 2023. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns* 4, 9 (2023), 100804. doi:10.1016/j.patter.2023.100804
- [9] Zixuan Ke, Yifei Ming, Austin Xu, Ryan Chin, Xuan-Phi Nguyen, Prathyusha Jwalapuram, Jiayu Wang, Semih Yavuz, Caiming Xiong, and Shafiq Joty. 2026. MAS-Orchestra: Understanding and Improving Multi-Agent Reasoning Through Holistic Orchestration and Controlled Benchmarks. *arXiv preprint arXiv:2601.14652* (2026).

- [10] John M. Lachin. 1992. Power and Sample Size Evaluation for the McNemar Test with Application to Matched Case-Control Studies. *Statistics in Medicine* 11, 9 (1992), 1239–1251. doi:10.1002/sim.4780110909
- [11] Simone Leo, Michael R. Crusoe, Laura Rodríguez-Navas, Raúl Sirvent, Alexander Kanitz, Paul De Geest, Rudolf Wittner, Luca Pireddu, Daniel Garijo, José M. Fernández, Iacopo Colonnelli, Matej Gallo, Tazro Ohta, Hirotaka Suetake, Salvador Capella-Gutierrez, Renske de Wit, Bruno P. Kinoshita, and Stian Soiland-Reyes. 2024. Recording Provenance of Workflow Runs with RO-Crate. *PLOS ONE* 19, 9 (2024), e0309210. doi:10.1371/journal.pone.0309210
- [12] Junyou Li, Qin Zhang, Yangbin Yu, Qiang Fu, and Deheng Ye. 2024. More Agents Is All You Need. *Transactions on Machine Learning Research (TMLR)* (2024). arXiv:2402.05120.
- [13] Quinn McNemar. 1947. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika* 12, 2 (1947), 153–157. doi:10.1007/BF02295996
- [14] Brian A. Nosek, Charles R. Ebersole, Alexander C. DeHaven, and David T. Mellor. 2018. The preregistration revolution. *Proceedings of the National Academy of Sciences* 115, 11 (2018), 2600–2606. doi:10.1073/pnas.1708274114
- [15] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. arXiv:2307.07924.
- [16] Stian Soiland-Reyes, Peter Sefton, Mercè Crosas, Leyla Jael Castro, Frederik Copen, José M. Fernández, Daniel Garijo, Björn Grüning, Marco La Rosa, Simone Leo, Eoghan Ó Carragáin, Marc Portier, Ana Trisovic, RO-Crate Community, Paul Groth, and Carole Goble. 2022. Packaging Research Artefacts with RO-Crate. *Data Science* 5, 2 (2022), 97–138. doi:10.3233/DS-210053
- [17] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. 2019. in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes. In *28th USENIX Security Symposium*. USENIX Association, 1393–1410.
- [18] Dat Tran and Douwe Kiela. 2026. Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets. *arXiv preprint arXiv:2604.02460* (2026).
- [19] Dennis Ulmer, Elisa Bassignana, Max Müller-Eberstein, Daniel Varab, Mike Zhang, Rob van der Goot, Christian Hardmeier, and Barbara Plank. 2022. Experimental Standards for Deep Learning in Natural Language Processing Research. In *Findings of the Association for Computational Linguistics: EMNLP 2022*. 2673–2692. arXiv:2204.06251 doi:10.18653/v1/2022.findings-emnlp.196
- [20] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryan W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155* (2023).